

POLITECNICO DI MILANO

1863

Department of Electronics, Information and Bioengineering

Embedded and Edge Artificial Intelligence

A.Y. 2025 – 26

Project Report

Doppler-Range UWB Radar Sensing with a TinyML Network for Indoor Human Localization

By

Marco Orlando Conte (10826144)

Sneharajalakshmi Palanisamy (11132155)

1 Introduction

The goal of the project is to estimate the 2D position of up to four people inside a room, starting from raw Ultra-Wideband (UWB) radar signals, with a model that fits the hardware constraints of an ESP32-S3 microcontroller.

1.1 Room and sensor configuration

The acquisition room measures 4.8×7.2 m. Six TSRR250 UWB radars are placed at fixed positions around the perimeter, at 1.2 m height, all pointing towards the centre of the room. Using the coordinate system of the dataset (origin at the bottom-left corner, x to the right, y upward), the radar positions are:

Radar	x (m)	y (m)
SR250_1	2.4	0.0
SR250_2	4.8	1.8
SR250_3	4.8	5.4
SR250_4	2.4	7.2
SR250_5	0.0	5.4
SR250_6	0.0	1.8

Each radar has 3 antennas and reports a CIR of 120 range bins per antenna, sampled at 25 Hz. One range bin corresponds to approximately 0.15 m.

We decided to use all six radars. The room is observed from six different directions, so a person that is radially far from one radar (poor range resolution of the reflection) is close and well resolved for the radar on the opposite wall. Angular diversity also helps to separate people standing at similar distances from a single sensor. The cost of this choice is a larger input tensor, but as shown in Section 6 the resulting model still fits comfortably inside the Flash and SRAM budgets, so there was no reason to discard sensors.

1.2 Input data and task

The dataset contains 24 acquisition windows of 300 s each (7500 frames per window, 180 000 frames in total). Each window is stored as a `.npz` file with the complex CIR array `radar_cir_iq` of shape $(T, 6, 3, 120, 2)$, the ground truth positions `people_xy` of shape $(T, 4, 2)$, a validity mask `people_mask` of shape $(T, 4)$ and the timestamps. The scenarios range from an empty room to four people, with combinations of random walks, serpentine walks and seated subjects.

At evaluation time the inference script receives a single `.npy` file of shape $(T, 6, 3, 120, 2)$ with no ground truth and outputs one JSON object per frame, containing the frame index and the list of estimated positions (from 0 to 4 people).

1.3 Hardware and software constraints

The model was specifically built to fit in the ESP32-S3 microcontroller. The project assignment specifies the following hardware and software constraints:

- Model file size $<$ 800 KB
- Tensor arena (SRAM) $<$ 300 KB
- INT8 quantization recommended

- No LSTM/GRU/RNN/CUSTOM/Flex operations
- NumPy $\geq$ 2.0.0
- SciPy $\geq$ 1.10.0
- TensorFlow $\geq$ 2.13.0

The constraints are verified by the official `evaluate_constraint.py` script provided with the project assignment.

1.4 Evaluation metric

Performance is measured by `evaluate_performance.py` as a global F1 detection score. For each frame, the predicted positions are matched to the ground truth using the Hungarian algorithm. A match counts as a true positive only if the distance between the predicted and ground-truth position is at most 1.0 m. Unmatched predictions are counted as false positives, while unmatched ground-truth persons are counted as false negatives.

The final evaluation is performed on a separate acquisition session that is never seen during training. Consequently, all design choices were made with generalisation in mind rather than fitting the training windows.

2 Data split

The two acquisition windows recorded with the room empty (windows 22 and 23) are excluded from training and validation: they carry no localisation labels and are used only to build the background template described in Section 3. Of the remaining 22 windows, four are held out for validation (windows 3, 9, 14 and 18, roughly every fifth window) and the other 18 are used for training.

The split is done at the level of entire acquisition windows, not of individual frames. Consecutive frames at 25 Hz are almost identical, so a random frame-level split would place near-duplicates of the training frames into the validation set and the validation score would be optimistic. Holding out whole 300 s recordings keeps the validation data temporally disjoint from the training data, which is closer to the situation of the final unseen session. The four validation windows were picked spread across the dataset, similarly in spirit to a stratified split, so that different occupancy levels and movement patterns are represented on both sides.

3 Preprocessing

The raw CIR is dominated by static reflections (walls, furniture, direct antenna coupling) that are orders of magnitude stronger than the reflections from people. The preprocessing stage removes this static component and summarises each range bin with a small set of features that describe how much, and how fast, the signal in that bin is moving.

3.1 Background subtraction in the complex domain

The empty-room windows 22 and 23 (15 000 frames in total) are averaged, per radar, antenna and range bin, into a complex background template of shape $(6, 3, 120, 2)$. This template is subtracted from every incoming frame before any other operation. Background subtraction is a sound choice in this setting because the room and the radar installation are fixed: every evaluation session, including the final unseen

one, is acquired in the same room with the same static scene, so a template estimated once from the empty-room recordings remains valid for any new window.

The subtraction is done on the I/Q components, not on the magnitude. A person standing near a strong static reflector changes mainly the phase of the received sum, and two complex contributions can partially cancel: subtracting magnitudes would lose this information, while the complex residual isolates exactly the contribution added by the moving targets. The template is computed once from the training data and frozen; it is embedded into `code.py` so that the inference script applies exactly the same subtraction. This embedded calibration blob (background template plus the normalization statistics of Section 3.5) is also the reason why `code.py` comparatively large for a script.

3.2 Range bin selection

Only the 60 bins from index 5 to index 64 are kept (slice `5:65` of the 120 raw bins). The initial five bins (indices 0 to 4) contain the transmitter-to-receiver direct coupling, which is not informative and is very strong even after background subtraction, so they are dropped. The upper limit follows from the room geometry: the first retained bin sits at $5 \times 0.15 = 0.75$ m and the last one at $64 \times 0.15 = 9.60$ m, so the retained interval covers distances up to 9.6 m from each radar, beyond the room diagonal ($\sqrt{4.8^2 + 7.2^2} \approx 8.65$ m). We kept this margin on purpose: cropping closer to the diagonal would save a few input columns, but a person near a far corner could then fall outside the kept bins and become impossible to detect, which would directly reduce the recall.

3.3 Sliding windows and Doppler features

A single frame contains limited temporal information and is often insufficient to reliably distinguish a person from residual clutter; what characterises a person is motion over time. The stream is therefore processed in sliding windows of 64 frames (2.56 s at 25 Hz). During training the windows advance with a stride of 16 frames and each window is labelled with the ground truth at its central frame.

For each window, six features are extracted per radar, antenna and range bin: the mean and standard deviation of the residual magnitude, three Doppler-band energies (low, mid, high), and a CA-CFAR contrast.

The first two features summarise the residual magnitude of a each window of size 64 frames directly in the time domain: the mean captures the average return strength over the window, while the standard deviation captures how much that return fluctuates: a coarse, cheap indicator of motion before any frequency analysis is applied.

The core of the feature extraction is the slow-time spectrum of the residual. For a given radar, antenna and range bin, let r_t, $t = 0, \ldots, 63$ be the complex background-subtracted samples of the window. The discrete Fourier transform along slow time and its power spectrum are

$$F_k = \sum_{t=0}^{63} r_t \, e^{-j2\pi kt/64}, \qquad P_k = \frac{|F_k|^2}{64},$$

and the energy of a Doppler band $[k_0, k_1]$ is

$$E_{[k_0,k_1]} = \sqrt{\sum_{k=k_0}^{k_1} (P_k + P_{64-k})},$$

where each positive-frequency bin is summed with its mirrored negative-frequency bin, so that motion towards and away from the radar contributes equally. With 64 samples at 25 Hz the frequency resolution is $25/64 \approx 0.39$ Hz, and three bands are extracted:

- *low*, bins 1–2 (0.39–0.78 Hz): breathing and micro-motion, which is what makes seated, nearly still people detectable;
- *mid*, bins 3–8 (1.17–3.13 Hz): body sway and slow walking;
- *high*, bins 9–20 (3.52–7.81 Hz): gross limb motion during faster movement.

Bin 0 (the DC component, i.e. whatever static residual survives the background subtraction) is deliberately excluded from all bands. Two further channels are the mean and the standard deviation of the residual magnitude $|r_t|$ over the window.

The sixth channel is a cell-averaging CFAR contrast computed along the range axis on the mean-magnitude profile x_i. With $T = 4$ training cells and $G = 2$ guard cells per side, the local noise floor at bin i is the average of the training cells excluding the guard cells,

$$\text{noise}_i = \frac{1}{2T}\left(\sum_{|j-i|\le T+G} x_j - \sum_{|j-i|\le G} x_j\right), \qquad \text{contrast}_i = \frac{x_i}{\text{noise}_i + \varepsilon}.$$

In the code both sums are computed with two uniform box filters (`uniform_filter1d` from SciPy), so the whole contrast map costs two passes over the array. The contrast acts as an adaptive whitening: a bin containing a person is compared against its own local clutter floor, so a weak reflection in a clean region stands out just as much as a strong reflection in a cluttered one.

The result is a tensor of shape $(6, 3, 60, 6)$ per window: 6 radars, 3 antennas, 60 range bins and 6 features. Compared to feeding the raw I/Q window (which would be $64 \times 6 \times 3 \times 120 \times 2$ values), this representation is about 40 times smaller, which matters both for the model input size and for the memory used by the preprocessing code.

3.4 Visualization

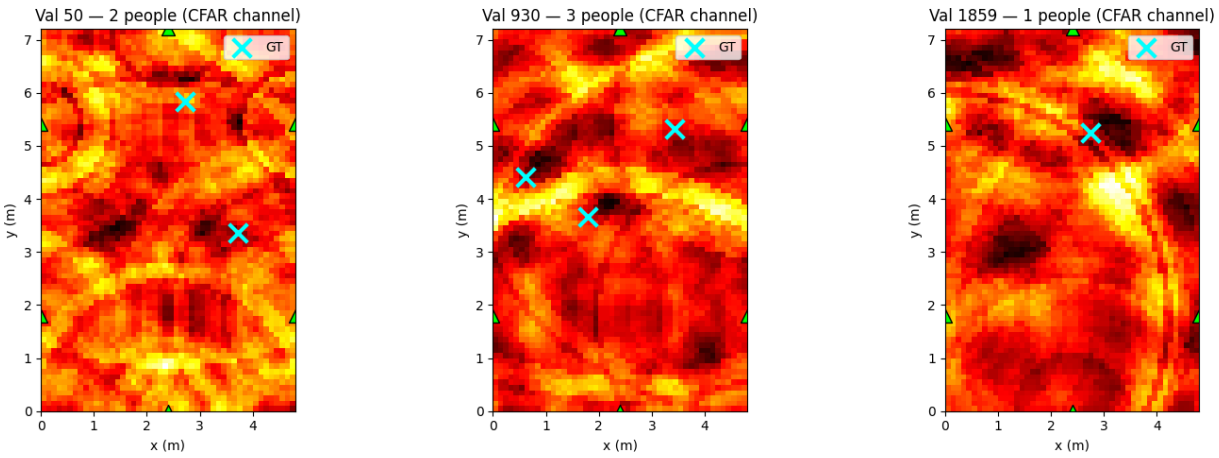


Figure 1: Heat map representation and sanity check.

Before training we ran a physics sanity check on this representation, shown in Figure 1: the CFAR channel of each radar is back-projected onto a grid of the room (each grid cell reads the range bin at its distance from the radar, using the 0.15 m bin size and the known radar positions) and the resulting map is overlaid with the ground truth. In the heat-map, the cyan crosses represent the ground-truth positions and the green triangles represent the six radars in the room. The bright intersections of the range arcs land on the ground truth across different occupancy levels, confirming that the radar geometry, the bin cropping and the feature extraction are mutually consistent before any learning takes place.

3.5 Normalization

The six feature channels have substantially different numerical ranges, with the Doppler-energy features spanning several orders of magnitude more than the magnitude-based ones, so each value goes through $\log(1+x)$ compression followed by Z-score normalization. The mean and standard deviation are computed per position on the training set only, frozen, and embedded in `code.py` together with the background template. Recomputing the statistics at inference time would let the input distribution drift with the session being evaluated, creating a mismatch with what the network saw during training.

3.6 Room symmetry augmentation

The room layout is symmetric with respect to both the vertical and the horizontal axis, and the six radars map onto each other under these reflections. This allows a cheap and physically consistent augmentation: mirroring the labels ($x \mapsto 4.8 - x$ or $y \mapsto 7.2 - y$) while permuting the radar axis of the features accordingly (for the horizontal flip the radar order becomes $[0,5,4,3,2,1]$, for the vertical flip $[3,2,1,0,5,4]$). Both flips are involutions, which wich was checked numerically. Applying the two flips triples the training set from 8370 to 25 110 windows without acquiring new data. The augmentation is applied to the training set only: neither the validation set nor the inference input is ever augmented. Unlike stochastic image augmentation, both reflections are generated deterministically for every training sample, so the augmented set is fixed and fully reproducible.

4 Model architecture

The network is a small 2D convolutional residual network with squeeze-and-excitation (SE) blocks, built only from layers supported by TensorFlow Lite for Microcontrollers: Conv2D, BatchNormalization, ReLU, MaxPool2D, GlobalAveragePooling, Dense, Add, Multiply and the sigmoid used inside the SE gate. No recurrent layers are used. A GaussianNoise layer ($\sigma = 0.05$) is applied to the normalized input during training only, as a regulariser, and is inactive at inference.

The input $(6,3,60,6)$ is first rearranged into a $(60,6,18)$ image: the 60 range bins become the first spatial axis, the 6 radars the second one, and the 3 antennas with their 6 features are stacked into 18 channels. This layout lets the convolutions correlate range profiles across radars, which is where the geometric information about position lives, while the antenna and feature dimensions are treated as channels. The initial 1×1 convolution of the stem mixes these antenna-feature channels while preserving the spatial arrangement of range bins and radars.

The body of the network is:

Stage	Layers	Output shape
Stem	1×1 Conv (48) + BN + ReLU + MaxPool $(2,1)$	$(30,6,48)$
Block 1	SE-Res block, 48 filters, dilation 1 + MaxPool $(2,1)$	$(15,6,48)$
Block 2	SE-Res block, 96 filters, dilation 2 + MaxPool $(2,1)$	$(7,6,96)$
Block 3	SE-Res block, 96 filters, dilation 3	$(7,6,96)$
Block 4	SE-Res block, 96 filters, dilation 1	$(7,6,96)$
Head	GAP + Dense 96 + Dropout 0.3 + Dense 12	$(4,3)$

Each SE-Res block contains two 3×3 convolutions with batch normalization, a squeeze-and-excitation gate and a residual connection with a 1×1 projection when the number of channels change. The SE gate is a global average pooling followed by two Dense layers, one that reduces the channels by a factor 4 with ReLU and one that restores them with a sigmoid, whose output multiplies the feature maps channel by channel. The dilations are applied along the range axis only, so deeper blocks increase the receptive field

along the range axis without increasing the parameter count or further reducing the spatial resolution; pooling is likewise applied only along the range axis, because the radar axis has just 6 positions and collapsing it early would mix the sensors too soon. The SE gates let the network reweight channels per sample, emphasising the Doppler band that is most informative for the current scene: when the subjects are seated, most of the human signal falls in the low breathing band, and the gate can raise its weight relative to the walking bands.

The output is a fixed set of 4 slots, each with three values: the x and y coordinates and a presence logit. A frame with fewer than 4 people is simply represented by slots whose presence probability is low.

5 Loss function and matching

The main difficulty of the slot formulation is that the ground truth persons have no defined order: slot 2 predicting person 1 is just as correct as slot 1 predicting person 1. The loss must therefore be invariant to permutations of the slots.

For each sample, the loss is computed for all $4! = 24$ permutations π of the prediction slots, and the minimum is kept:

$$\mathscr{L} = \min_{\pi} \left[\frac{\sum_k m_k \, \|\hat{p}_{\pi(k)} - g_k\|^2}{\sum_k m_k + \varepsilon} \; + \; 0.7 \cdot \frac{1}{4} \sum_k \mathrm{BCE}\big(m_k, \hat{\ell}_{\pi(k)}\big) \right]$$

where g_k are the ground truth positions, $m_k \in \{0,1\}$ the validity mask, $\hat{p}$ the predicted coordinates and $\hat{\ell}$ the presence logits. The first term is the mean squared error (MSE) of the position, computed over the valid persons only, so slots assigned to absent persons are not penalised on their coordinates; being an MSE and not a root mean squared error, it grows quadratically with the distance and pushes hard on large mistakes. The second term is a binary cross-entropy that pushes the presence logits towards the mask, weighted by 0.7 so that neither term dominates.

Taking the minimum over all 24 permutations is an exhaustive optimal assignment: for 4 slots it computes exactly the same matching that the Hungarian algorithm would find, but as a small fixed tensor operation (a gather over a constant 24×4 permutation table) that runs efficiently on GPU inside the training graph. The Hungarian algorithm itself (via `scipy.optimize.linear_sum_assignment`) is used outside the training loop, in our validation evaluator, which replicates the official scoring: optimal matching per frame, true positive only within 1.0 m, and global precision, recall and F1. Using the same matching logic for training, model selection and final scoring keeps the whole pipeline optimised for the actual metric.

The presence component is intentionally left as a raw logit rather than being passed through a sigmoid activation inside the network. The loss uses the numerically stable sigmoid cross-entropy with logits formulation, and at inference the sigmoid is applied outside the model, in `code.py`; this also keeps one operation out of the quantized graph.

Training uses Adam with learning rate 10^{-3} and gradient norm clipping at 1.0, batch size 128, for at most 120 epochs with early stopping (patience 12 on the validation loss), learning rate halving on plateau (patience 5, down to 10^{-5}) and checkpointing of the best epoch.

Threshold calibration is performed after restoring the best validation checkpoint: the presence threshold is swept from 0.05 to 0.95 on the validation set, and the value maximising the F1 is 0.87, which trades a little recall for precision and noticeably improves the per-frame count accuracy (from 0.881 at threshold 0.5 to 0.938). This threshold is the one hard-coded in `code.py`.

6 Quantization and deployment

6.1 Post-training INT8 quantization

The trained Keras model is converted to TFLite with full integer post-training quantization (PTQ): weights and activations are quantized to INT8, and the input and output tensors are also INT8. A representative dataset of 200 samples drawn from the training set is passed to the converter; it is used only to estimate the activation ranges during conversion, it does not participate in any optimisation and it does not modify the learned weights, which is why a small subset is sufficient. The converter is restricted to the `TFLITE_BUILTINS_INT8` op set, which guarantees that no float fallback or Flex op ends up in the model.

We chose PTQ rather than quantization-aware training because it requires no change to the training loop.

The effect of quantization on size and accuracy is summarised here:

	float32	INT8	change
Model size	2.03 MB	577.9 KB	3.6× smaller
Validation F1 (th = 0.87)	0.8597	0.8535	−0.006
Count accuracy	0.9376	0.9355	−0.002

The INT8 model was re-evaluated on the full validation set through the TFLite interpreter, so the drop of 0.006 F1 is measured end to end and not estimated. The small reduction indicates that the learned feature representations are robust to 8-bit quantization, especially PTQ because it requires no change to the training loop. The same validation-calibrated presence threshold (0.87) was retained for the INT8 model.

6.2 Constraint check

Running the official checker on the submitted `model.tflite` gives:

Check	Limit	Measured	Result
TFLite validity	loads and allocates	input $(1,6,3,60,6)$, output $(1,4,3)$, int8	PASS
Model file size	$<$ 800 KB	577.9 KB	PASS
Activation arena	$<$ 300 KB	212.4 KB	PASS
Forbidden ops	none	none detected	PASS
Quantization	INT8 recommended	full INT8	PASS

The activation arena estimate of 212.4 KB is an upper bound, since TFLM overlaps buffers that are not alive at the same time, so the real arena on device would be at most this value. Both hard limits are met with margin (72% of the Flash budget, 71% of the SRAM budget).

6.3 Inference script

`code.py` uses only NumPy, SciPy and TensorFlow, plus standard library modules; `uniform_filter1d` and the temporal smoothing of the presence probabilities described below. The background template and the normalization statistics are embedded in the script as a compressed base64 blob, so the preprocessing at inference is numerically identical to the one used to build the training set: the extracted features are first compressed with $\log(1+x)$ and then normalized with the frozen Z-score statistics, exactly as during training.

For each output frame the script extracts the features from the 64-frame window centred on that frame (windows are clamped at the borders of the recording, and recordings shorter than 64 frames are padded

by repeating the last frame). Before inference, the normalized floating-point features are quantized using the scale and zero point stored in the TFLite model, $q = \text{round}(x/\text{scale} + \text{zero_point})$, and fed to the interpreter. This is a stride-1 sliding window, so every frame gets its own prediction as required by the output format. The four presence probabilities are then smoothed over time with a 5-frame moving average per slot, which removes single-frame flickers of the detections, and thresholded at the calibrated value 0.87. Predicted coordinates are clipped to the room bounds $[0, 4.8] \times [0, 7.2]$, since the format validation of the evaluator rejects any coordinate outside the room.

7 Results and discussion

7.1 Validation results

On the four validation windows the INT8 model reaches:

Metric	Value
F1 (threshold 0.87)	0.8535
Precision (float, same threshold)	0.8561
Recall (float, same threshold)	0.8634
Count accuracy	0.9355
Median localisation error	0.357 m
P90 localisation error	0.785 m

The localisation errors are computed only over correctly matched predictions, i.e. pairs within the 1.0 m radius: they describe how accurate the detections already counted as correct by the F1 are, which is why they sit well below the matching threshold.

The per-occupancy breakdown of the F1: 1.00 with the empty room, 0.996 with one person, 0.987 with two, 0.778 with three and 0.827 with four. The system is essentially perfect up to two people, and performance decreases in the three and four person scenarios, where the radar observations become substantially more complex because multiple people contribute overlapping reflections. This is the expected failure mode of the approach and is where most of the residual error concentrates.

The precision/recall balance at the calibrated threshold is nearly symmetric, so the model is neither systematically hallucinating people nor systematically missing them. Figure 2 shows the training behaviour: the loss converges smoothly, early stopping halts the training well before the 120-epoch budget, and the gap between the training and validation curves stays moderate. The presence classification itself is strong: with Hungarian-matched labels, the presence logits reach a ROC AUC of 0.996 on the training set and 0.948 on validation. The gap between the two curves is the ordinary generalisation gap, and the validation curve still rises steeply near the origin, which is what allows the high calibrated threshold of 0.87 to keep the recall at 0.86 while filtering out most spurious detections.

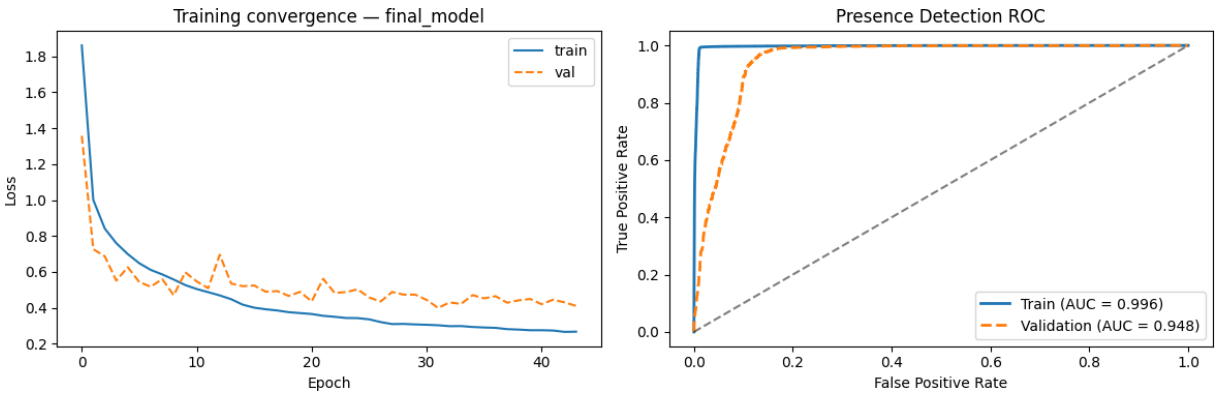


Figure 2: Training convergence and presence detection ROC.

7.2 Stability with respect to the data split

To make sure the reported numbers do not depend on the particular choice of validation windows, we repeated the whole training with a 5-fold cross-validation over the 22 non-empty acquisitions: the windows were shuffled once (seed 42) and partitioned into five groups of 4 or 5, each group serving in turn as validation set while the remaining windows were used for training. The optimal presence threshold was calibrated independently on each validation fold before computing the reported F1 scores, exactly as in the main pipeline, which is why the calibrated threshold differs from fold to fold. The results are:

Fold	Validation windows	F1 @0.5	F1 calibrated	th	Count acc.	Median err.
0	[0, 1, 8, 13, 15]	0.8424	0.8428	0.70	0.9247	0.385 m
1	[3, 4, 5, 11, 20]	0.8574	0.8586	0.65	0.9196	0.363 m
2	[12, 16, 17, 18]	0.8714	0.8917	0.75	0.7425	0.296 m
3	[2, 7, 9, 21]	0.7691	0.7974	0.75	0.7532	0.525 m
4	[6, 10, 14, 19]	0.8243	0.8252	0.55	0.7876	0.451 m

Figure 3 shows the same results graphically: for each fold, the F1 at the default and at the calibrated threshold with the cross-fold mean and the one-standard-deviation band, the per-frame count accuracy, and the fold-to-fold spread of the two F1 scores.

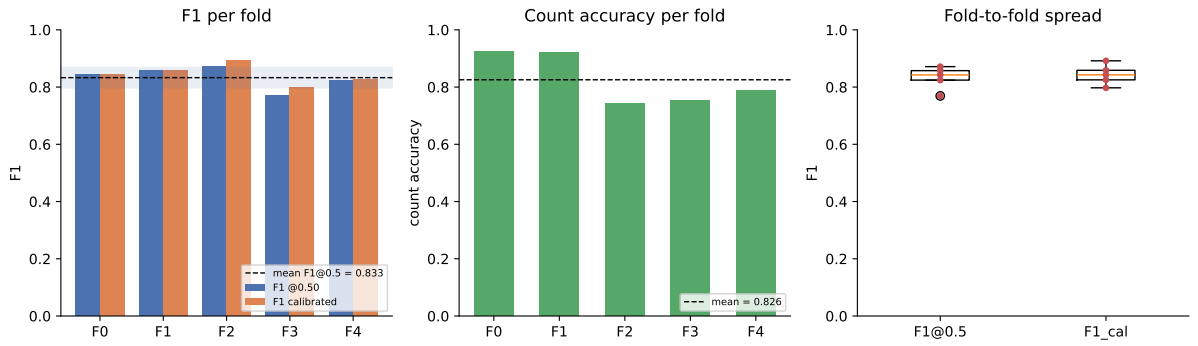


Figure 3: 5-fold cross-validation of the final model.

Across the five folds the calibrated F1 is 0.843 ± 0.032 (mean $\pm$ standard deviation), with precision 0.814 ± 0.041 and recall 0.854 ± 0.050. No fold collapses, and the split used for the submitted model

falls within one standard deviation of the cross-validation mean, so the figure reported in the previous subsection is representative of the method and not an artifact of a lucky choice of validation windows. The spread between the best fold (0.89) and the worst one (0.80) mainly reflects which scenarios end up in validation: folds whose held-out windows contain more of the crowded three and four person recordings score lower, consistently with the per-occupancy breakdown discussed above. The optimal presence threshold also moves between 0.55 and 0.75 depending on the fold, which confirms that calibrating it on the validation data, rather than fixing it a priori, is the right procedure.

7.3 End-to-end check on the provided example

As a final sanity check the whole submission pipeline was run exactly as instructed: `code.py` on `input_test.npy`, then `evaluate_performance.py` against the reference output, then `evaluate_constraint.py` on the model. The output passed the format validation and scored F1 = 0.9969 (precision 0.9984, recall 0.9955, RMSE 0.246 m, count accuracy 99.4% over 5000 frames), and the constraint check passed all five tests. These figures are much higher than the validation ones because the example input is derived from the training data; as stated in the assignment it is meant for format verification, not for performance benchmarking, so the numbers we consider representative of unseen data are the validation ones of the previous subsections.

8 Further improvements

The clearest margin for improvement is in the three and four person scenarios. A few directions we consider promising:

- **Heatmap output instead of slots.** Predicting an occupancy heatmap over a grid of the room, followed by peak extraction, would remove the fixed-slot assignment problem entirely and might separate nearby people better. The cost is a larger output tensor and a decoding step in `code.py`, and the arena usage would have to be re-checked.
- **Longer temporal context.** The 64-frame window sees 2.56 s of motion. A second-stage temporal filter on the predicted trajectories (for example a simple constant-velocity tracker with Hungarian association between consecutive frames) could recover detections that flicker below the threshold and smooth the position estimates, using only NumPy and SciPy at inference.
- **Threshold per occupancy.** The single global presence threshold is a compromise between the sparse and the crowded scenes. Estimating the person count first and adapting the threshold accordingly could recover part of the recall lost in the four person case.
- **Quantization-aware training.** Quantization-aware training could be explored if future architectures become more sensitive to quantization, although the current post-training quantization incurs only a small accuracy drop (0.006 F1).

None of these changes conflicts with the hardware constraints, since the current model uses less than three quarters of both the Flash and the SRAM budgets.

References

[1] F. Fioranelli, E. Rufas, and A. Yarovoy. Multiple People Detection & Localization with Multi-static UWB Radar in Vehicle Cabin. *Proceedings of the IEEE International Workshop on Antenna Technology (iWAT)*, pp. 277–280, 2024.

[2] M. Pavan, A. Caltabiano, and M. Roveri. TinyML for UWB-radar based Presence Detection. *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2022.

[3] S. Campaña-Bastidas, M. Espinilla-Estévez, and J. Medina-Quero. Review of Ultra Wide Band (UWB) for Indoor Positioning with Application to the Elderly. *Proceedings of the 55th Hawaii International Conference on System Sciences (HICSS)*, pp. 2145–2154, 2022.

[4] M. A. Richards. *Fundamentals of Radar Signal Processing*. McGraw-Hill, 2nd edition, 2014.

[5] M. Roveri. Lecture Slides and Lab Exercises of the Course *Edge Evolutionary Architecture Intelligence (EEAI)*. Politecnico di Milano, 2026.